

Universidad Autónoma de Guadalajara
Campus Tabasco

Departamento Académico de
Ciencias Sociales y Humanidades

[Tipo de Actividad: Ensayo, Resumen, etc.]
Practica 1.

Programa
Ing. De software y Minería de Datos

Asignatura
Programación Orientada a Objetos.

Presenta
Daniel Ernesto Garrido Sánchez.
Matrícula 5211038

Docente
Manuel Balderas Victoria

Villahermosa, Tabasco, México
11/09/2026

Introducción

Esta tarea integra 5 ejercicios cortos que demandan escribir, correr y depurar código propio, además de documentarlo bajo estándares del mundo laboral: aportando evidencia de su funcionamiento, manteniendo el código en un repositorio público y sumando una reflexión honesta sobre las complicaciones enfrentadas.

- Link repositorio: https://github.com/DanielErnestoGS/POO_UAG

Ejercicio 1: Calculadora de consola

Programa que lea dos números y un operador (+, -, *, /) por entrada estándar y muestre el resultado, usando tipos primitivos y operadores. Debe validar la división entre cero.

Código:

```
#include <iostream>

class Calculadora {
private:
    float primerNumero = 0.0;
    float segundoNumero = 0.0;
    float resultado = 0.0;
    char operador = ' ';

public:
    void sumar() {
        resultado = primerNumero + segundoNumero;
        mostrarResultado();
    }

    void restar() {
        resultado = primerNumero - segundoNumero;
        mostrarResultado();
    }

    void multiplicar() {
        resultado = primerNumero * segundoNumero;
        mostrarResultado();
    }

    void dividir() {
        if (segundoNumero != 0) {
            resultado = primerNumero / segundoNumero;
            mostrarResultado();
        } else {
            std::cout << "Error: No se puede dividir entre cero.
```

```
:(\n";
        }
    }

    void ingresarDatos() {
        std::cout << "Ingresa el primer numero: ";
        std::cin >> primerNumero;

        std::cout << "Ingresa el operador (+, -, *, /): ";
        std::cin >> operador;

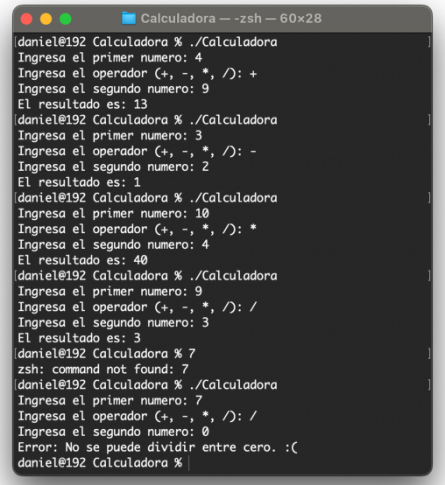
        std::cout << "Ingresa el segundo numero: ";
        std::cin >> segundoNumero;

        // Ejecuta la operacion segun el operador ingresado
        switch (operador) {
            case '+': sumar(); break;
            case '-': restar(); break;
            case '*': multiplicar(); break;
            case '/': dividir(); break;
            default:  std::cout << "Operador no valido.\n"; break;
        }
    }

    void mostrarResultado() {
        std::cout << "El resultado es: " << resultado << "\n";
    }
};

int main() {
    Calculadora miCalculadora;
    miCalculadora.ingresarDatos();
    return 0;
}
```

Captura de pantalla de la ejecución.



Liga al archivo (commit) de GitHub.

https://github.com/DanielErnestoGS/POO_UAG/commit/c71164685ad639ee66d26adfa7dbf865d22d198b

Reflexión

En la calculadora aprendí a hacer la abstracción de todo los métodos y atributos necesarios en el programa, también aprendí a utilizar los operadores para distintas funciones para realizar las operaciones.

Ejercicio 2: Clasificador con sentencias de control

Programa que reciba una calificación (0-100) y, usando if/else o switch, muestre si es NA, S, B, MB o E según el criterio de tu institución.

Código:

```
#include <iostream>
#include <string>

class ClasificadorCalificaciones {
private:
    int calificacionNumerica = 0;
    std::string calificacionSimbolica = "";

public:
    void convertir() {
        switch (calificacionNumerica) {
        case 0 ... 59:
            calificacionSimbolica = "NA";
            mostrarResultado();
            break;
        case 60 ... 69:
            calificacionSimbolica = "S";
            mostrarResultado();
            break;
        case 70 ... 79:
            calificacionSimbolica = "B";
            mostrarResultado();
            break;
        case 80 ... 89:
            calificacionSimbolica = "MB";
            mostrarResultado();
            break;
        case 90 ... 100:
            calificacionSimbolica = "E";
            mostrarResultado();
            break;
```

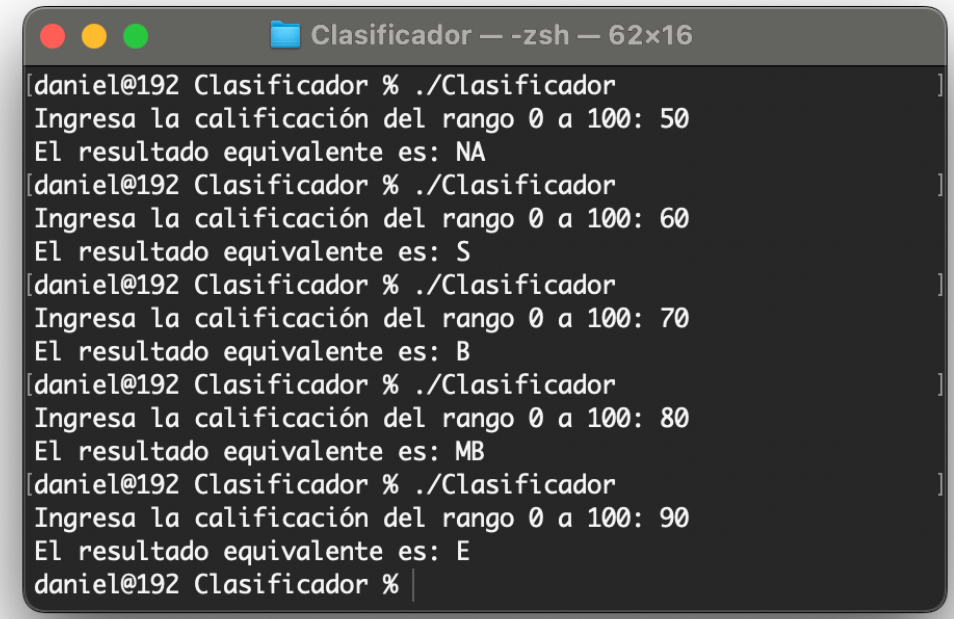
```
        default:
            std::cout << "Error: No se está en los rangos
permitidos :(\n";
            break;
        }
    }

    void ingresarDatos() {
        std::cout << "Ingresa la calificación del rango 0 a 100: ";
        std::cin >> calificacionNumerica;
        convertir();
    }

    void mostrarResultado() {
        std::cout << "El resultado equivalente es: " <<
calificacionSimbolica << "\n";
    }
};

int main() {
    ClasificadorCalificaciones clasificador;
    clasificador.ingresarDatos();
    return 0;
}
```

Captura de pantalla de la ejecución.



Liga al archivo (commit) de GitHub.

https://github.com/DanielErnestoGS/POO_UAG/commit/0e477ab592046867dd2a3636b047026a065a9a36

Reflexión

Aprendí a utilizar las sentencias para la toma de decisiones dependiendo de la calificación. Fue complicado al principio, pero me ayudo a entender mejor cómo funcionan las condiciones dentro de un programa.

Ejercicio 3: Procesador de cadenas

Programa que reciba una frase y muestre: número de palabras, número de vocales, y la frase invertida, usando funciones de manejo de cadenas.

Código:

```
#include <iostream>
#include <string>
#include <algorithm> // Necesario para std::reverse
#include <cctype>    // Necesario para std::tolower

class ProcesadorCadenas {
private:
    std::string cadena = "";

public:
    void mostrarNumeroPalabras() {
        if (cadena.empty()) {
            std::cout << "Numero de palabras: 0\n";
            return;
        }
        int espacios = 0;
        // Contamos el número de espacios para conocer cuántas
palabras hay
        for (char c : cadena) {
            if (c == ' ') {
                espacios++;
            }
        }
        int palabras = espacios + 1;
        std::cout << "Numero de palabras: " << palabras << "\n";
    }

    void mostrarNumeroVocales() {
        int vocales = 0;
```

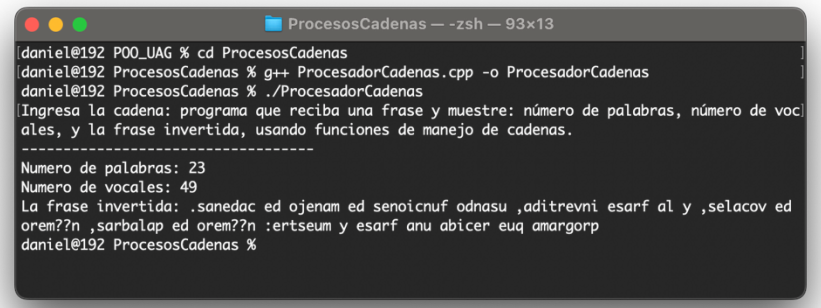
```
        for (char c : cadena) {
            // contamos las vocales, primero ponemos todo
minúscula.
            char letra = std::tolower(c);
            if (letra == 'a' || letra == 'e' || letra == 'i' ||
letra == 'o' || letra == 'u') {
                vocales++;
            }
        }
        std::cout << "Numero de vocales: " << vocales << "\n";
    }

    void mostrarFraseInvertida() {
        std::string cadenaInvertida = cadena;
        // Usamos la biblioteca algorithm para invertir el string.
        std::reverse(cadenaInvertida.begin(),
cadenaInvertida.end());
        std::cout << "La frase invertida: " << cadenaInvertida <<
"\n";
    }

    void ingresarCadena() {
        std::cout << "Ingresa la cadena: ";
        // std::getline permite capturar la frase completa
incluyendo espacios
        std::getline(std::cin, cadena);
    }
};

int main() {
    ProcesadorCadenas clasificador;
    clasificador.ingresarCadena();
    std::cout << "-----------------------------------\n";
    clasificador.mostrarNumeroPalabras();
    clasificador.mostrarNumeroVocales();
    clasificador.mostrarFraseInvertida();
    return 0;
}
```

Captura de pantalla de la ejecución.



Liga al archivo (commit) de GitHub.

https://github.com/DanielErnestoGS/POO_UAG/commit/3a569293c2104c42413927e95bc301f32126e45f

Reflexión

Aprendí a entender mejor como se pueden manejar frases y los caracteres, lo complicado fue tener que contar las palabras y poner las frases que se muestren al revés, porque se tiene que recorrer cada un carácter.

Ejercicio 4: Mi primera clase

Define una clase Estudiante con atributos (nombre, matrícula, promedio) y al menos dos métodos (por ejemplo, mostrar información y verificar si está en riesgo académico). Crea un objeto y demuestra que funciona.

Código:

```
// ESTUDIANTE.h ----------------------------------

#pragma once
#include <string>

class Estudiante {
private:
    std::string nombre;
    std::string matricula;
    float promedio;

public:
    Estudiante(std::string nombre, std::string matricula, float
promedio);

    void mostrarInformacion();
    void verificarRiesgoAcademico();
    float getPromedio() const;
    std::string getNombre() const;
};

// ESTUDIANTE.cpp ---------------------------------

#include <iostream>
#include "Estudiante.h"

Estudiante::Estudiante(
    std::string nombre,
```

```
    std::string matricula,
    float promedio
) : nombre(nombre), matricula(matricula), promedio(promedio) { }

void Estudiante::mostrarInformacion() {
    std::cout << "Nombre: " << nombre << "\n";
    std::cout << "Matricula: " << matricula << "\n";
    std::cout << "Promedio: " << promedio << "\n";
}

void Estudiante::verificarRiesgoAcademico() {
    if(promedio > 7) {
        std::cout << nombre << " NO se encuentra en riesgo
academico.\n";
    } else {
        std::cout << nombre << " se encuentra en riesgo
academico.\n";
    }
}

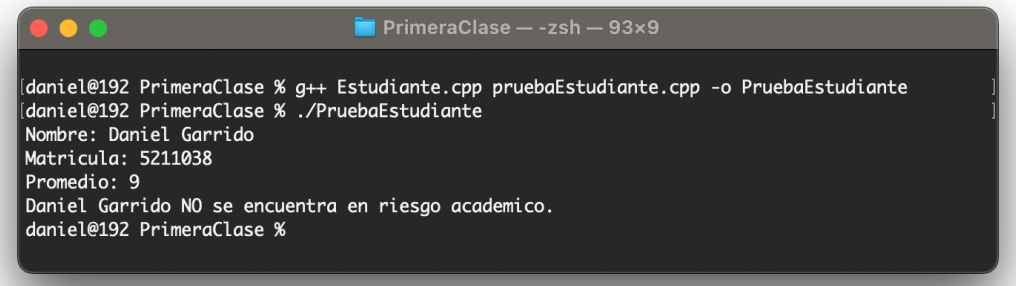
float Estudiante::getPromedio() const { return promedio; }
std::string Estudiante::getNombre() const { return nombre; }
```

// PruebaEstudiante.cpp ----------

```
#include <iostream>
#include "Estudiante.h"

int main() {
    Estudiante estudiante("Daniel Garrido", "5211038", 9.0);
    estudiante.mostrarInformacion();
    estudiante.verificarRiesgoAcademico();
    return 0;
}
```

Captura de pantalla de la ejecución.



Liga al archivo (commit) de GitHub.

https://github.com/DanielErnestoGS/POO_UAG/commit/e501b3dcecaaeeeac1e884597a0b9d7b1f312691

Reflexión

Este ejercicio fue un claro ejemplo y de lo que me ayudó mucho a entender la programación orientada a objetos, me mostró la manera en la que cada método y atributo se ve influenciado o afectado en el programa si no te organizas bien.

Ejercicio 5: Interacción entre objetos

Crea una clase Grupo que use un arreglo de objetos Estudiante (mínimo 3) y un método que calcule el promedio general del grupo.

Código:

// Grupo.cpp

```
#include <iostream>
#include <string>
#include <vector>
#include "../PrimeraClase/Estudiante.h"

class Grupo {
private:
    std::string nombre = "Clase POO UAG";
    std::string nombreMaestro = "Maestro";

    // Inicializamos el vector enviándole los parámetros requeridos
por el constructor
    std::vector<Estudiante> estudiantes = {
        Estudiante("Daniel Garrido", "5211038", 9.0),
        Estudiante("Ana Lopez", "5211039", 6.5),
        Estudiante("Carlos Perez", "5211040", 8.2),
        Estudiante("Maria Gomez", "5211041", 5.8),
        Estudiante("Jorge Ruiz", "5211042", 9.5)
    };

public:
    void mostrarListaEstudiantes() {
        std::cout << "--- Lista de Estudiantes ---\n";
        for (size_t i = 0; i < estudiantes.size(); i++) {
            std::cout << i + 1 << ". " <<
estudiantes[i].getNombre()
                      << " (Promedio: " <<
estudiantes[i].getPromedio() << ")\n";
        }
        std::cout << "----------------------------\n";
    }

    void mostrarPromedioGrupo() {
        if (estudiantes.empty()) {
            std::cout << "El grupo no tiene estudiantes.\n";
            return;
```

```
        }

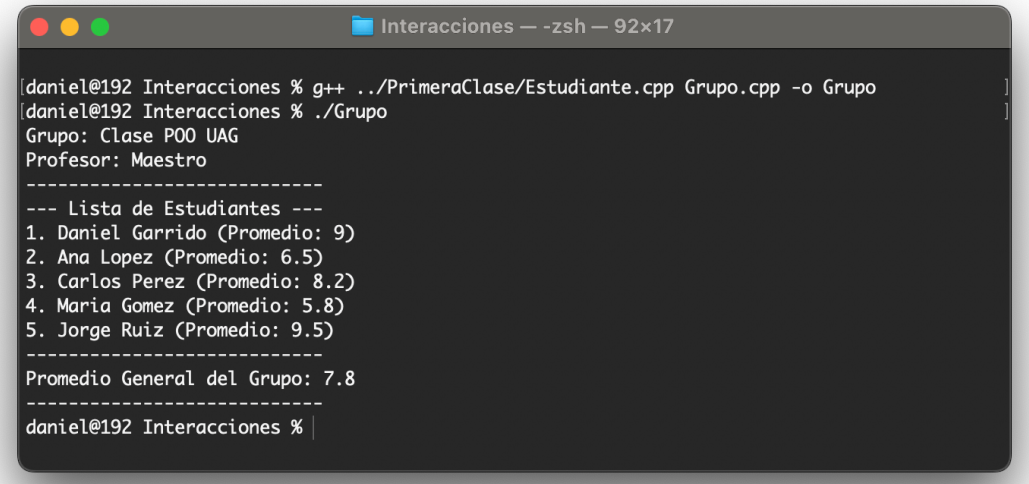
        float sumaPromedios = 0.0;
        for (const auto& est : estudiantes) {
            sumaPromedios += est.getPromedio();
        }

        float promedioGrupo = sumaPromedios / estudiantes.size();
        std::cout << "Promedio General del Grupo: " <<
promedioGrupo << "\n";
        std::cout << "---------------------------\n";
    }

    void mostrarDatosGrupo() {
        std::cout << "Grupo: " << nombre << "\n";
        std::cout << "Profesor: " << nombreMaestro << "\n";
        std::cout << "---------------------------\n";
        mostrarListaEstudiantes();
        mostrarPromedioGrupo();
    }
};

int main() {
    Grupo grupoDePOO;
    grupoDePOO.mostrarDatosGrupo();
    return 0;
}
```

Captura de pantalla de la ejecución.



Liga al archivo (commit) de GitHub.

https://github.com/DanielErnestoGS/POO_UAG/commit/957db103536c79966021edabb27c007dfef8c70a

Reflexión

Aprendí a entender la programación debe ser de manera organizaba y que mejor manera que usando las clases para definir cuales atributos y métodos realmente son importantes, a veces ahí atributos que son confidenciales por eso las clases deben de ser privadas y lo que no pues lo demás ira en clases públicas.

Conclusión.

Aprendí la importancia de 4 pilares de la programación, los atributos y métodos que se le tienen que ingresar y la diferencia que tienen uno tiene y el otro hace, lo complicado fue hacer el proceso de elección de los métodos y atributos incluyendo la división entre 0.

También aprendí lo importante que puede ser la toma de decisiones de un programa, fue confuso organizarlos de la manera correcta tuve que apoyarme con los conceptos aprendidos para lograr entenderlo mejor, me ayudo a entender que un buen orden puede mejorar tu forma de programar además de las condiciones.

En lo personal también algo muy complicado que sufrí fue tener que planearlo todo, el principal pilar de la programación fue la abstracción para mí, tener que elegir que necesito y que no para luego tomar lo que voy a usar a veces es complicado debido a que hay muchos datos que se pueden tomar en cuenta, además de que muchos cometemos el error de tocar el teclado antes de programar sin siquiera saber que quieres hacer exactamente.